OPinit 통합 분석

1. OPinit 개요

OPinit은 Initia Labs에서 개발한 Cosmos SDK 기반의 Optimistic Rollup 프레임워크입니다.
MiniEVM은 이 OPinit 스택을 활용하여 L2 체인을 구현합니다.

1.1 OPinit vs Optimism 비교

| 구분 | Optimism | OPinit |

|------|----------|--------|

| **기반 프레임워크** | Ethereum 기반 | Cosmos SDK 기반 |

| **합의 메커니즘** | Ethereum L1 보안 상속 | CometBFT + L1 보안 상속 |

| **가상머신** | EVM | EVM + Cosmos 모듈 |

| **브리지** | Ethereum ↔ Optimism | L1 ↔ L2 (IBC 기반) |

| **상태 증명** | Merkle Patricia Trie | Cosmos State Machine |

| **사기 증명** | Ethereum 기반 | IBC 기반 |

1.2 MiniEVM에서 사용하는 OPinit 컴포넌트

```
// OPinit 의존성

github.com/initia-labs/OPinit  v1.1.3

github.com/initia-labs/OPinit/api  v1.1.0
```

2. OPinit 모듈 구조

2.1 OPChild 모듈 (L2 체인용)

MiniEVM에서 사용하는 핵심 모듈:

```
// OPChild 모듈 임포트
opchild  "github.com/initia-labs/OPinit/x/opchild"
opchildkeeper  "github.com/initia-labs/OPinit/x/opchild/keeper"
opchildtypes  "github.com/initia-labs/OPinit/x/opchild/types"
opchildante  "github.com/initia-labs/OPinit/x/opchild/ante"
opchildlanes  "github.com/initia-labs/OPinit/x/opchild/lanes"
```

2.2 OPHost 모듈 (L1 체인용)

L1에서 L2를 관리하는 모듈:

```
// OPHost 타입 사용
ophosttypes  "github.com/initia-labs/OPinit/x/ophost/types"


// L1에서 L2로 토큰 예치 메시지
initiateTokenDepositMsg := ophosttypes.NewMsgInitiateTokenDeposit(
executorL1AddrStr,
bridgeId,
executorL2AddrStr,
sdk.NewCoin(INIT_DENOM, sdkmath.NewInt(0)),
```

```
hookMsg,

)
```

3. Launch Tools 통합

3.1 자동화된 L2 체인 배포

OPinit의 Launch Tools를 사용하여 L2 체인을 자동으로 배포합니다:

```
// Launch Tools 임포트

"github.com/initia-labs/OPinit/contrib/launchtools"

"github.com/initia-labs/OPinit/contrib/launchtools/steps"

"github.com/initia-labs/OPinit/contrib/launchtools/utils"


// 기본 배포 단계들

var  DefaultLaunchStepFactories = []launchtools.LauncherStepFuncFactory[*launcht

steps.InitializeConfig, // 설정 초기화

steps.InitializeRPCHelpers, // RPC 헬퍼 초기화

steps.InitializeGenesis, // 제네시스 초기화

steps.InitializeKeyring, // 키링 초기화

steps.RunAppWithPostAction(...), // 앱 실행

steps.EstablishIBCChannelsWithNFTTransfer(...), // IBC 채널 설정

steps.InitializeOpBridge, // OP 브리지 초기화

steps.SetBridgeInfo, // 브리지 정보 설정
```

```
steps.GetL1Height, // L1 높이 조회
steps.GetL2Height, // L2 높이 조회
CreateINITWrappedToken, // 래핑된 토큰 생성
steps.StopApp, // 앱 종료
}
```

3.2 브리지 초기화 과정

```
func  CreateINITWrappedToken(config *launchtools.Config) launchtools.LauncherSte
return  func(ctx launchtools.Launcher) error {
// 1. 브리지 ID 및 실행자 주소 설정
bridgeId := *ctx.GetBridgeId()
executorL1AddrStr := config.SystemKeys.BridgeExecutor.L1Address
executorL2AddrStr := config.SystemKeys.BridgeExecutor.L2Address
// 2. OP 브리지 후크 메시지 생성
hookMsg, err := generateOPBridgeHookMessage(ctx, bridgeId, operatorL2AddrStr, op
// 3. 토큰 예치 메시지 생성
initiateTokenDepositMsg := ophosttypes.NewMsgInitiateTokenDeposit(
executorL1AddrStr,
bridgeId,
executorL2AddrStr,
sdk.NewCoin(INIT_DENOM, sdkmath.NewInt(0)),
```

```
hookMsg,
)
// 4. L1에 트랜잭션 브로드캐스트
res, err := ctx.GetRPCHelperL1().BroadcastTxAndWait(...)
return nil
}
}
```

4. 브리지 시스템 통합

4.1 L1 → L2 토큰 예치

```
// L1에서 실행되는 토큰 예치 메시지
type MsgInitiateTokenDeposit struct {
Sender string // 발신자 주소
BridgeId uint64 // 브리지 ID
To string // L2 수신자 주소
Amount sdk.Coin // 예치할 토큰 양
Data []byte // 추가 데이터 (후크 메시지)
}
```

4.2 L2 → L1 토큰 출금

ERC20 Wrapper 컨트랙트에서 OPinit 메시지 생성:

```
function _op_withdraw(
string memory receiver,
address token,
uint amount
) internal view returns (string memory message) {
// OPinit 출금 메시지 구성
message = string(
abi.encodePacked(
'{"@type": "/opinit.opchild.v1.MsgInitiateTokenWithdrawal"',
',"amount": { "denom": "',
COSMOS_CONTRACT.to_denom(token),
'","amount": "',
Strings.toString(amount),
'"},"sender": "',
COSMOS_CONTRACT.to_cosmos_address(address(this)),
'","to": "',
receiver,
'"}'
)
);
}
```

4.3 L2 Denom 계산

```
// L1 토큰에 대응하는 L2 토큰 denom 계산

l2Denom := ophosttypes.L2Denom(bridgeId, INIT_DENOM)
```

5. OPChild 모듈 통합

5.1 모듈 등록

```
// app/modules.go에서 OPChild 모듈 등록

opchild.NewAppModule(app.appCodec, app.OPChildKeeper)
```

5.2 Keeper 초기화

```
// OPChild Keeper 생성

appKeepers.OPChildKeeper = opchildkeeper.NewKeeper(

appCodec,

runtime.NewKVStoreService(appKeepers.keys[opchildtypes.StoreKey]),

appKeepers.AccountKeeper,

appKeepers.BankKeeper,

appKeepers.OracleKeeper,

ante.CreateAnteHandlerForOPinit(appKeepers.AccountKeeper, txConfig.SignModeHandl

txConfig.TxDecoder(),

bApp.MsgServiceRouter(),

authorityAddr,
```

```
ac, vc, cc,
logger,
)
```

5.3 스테이킹 인터페이스 구현

```
// OPChild가 스테이킹 모듈 역할 수행
func (app *MinitiaApp) GetStakingKeeper() ibctestingtypes.StakingKeeper {
return  app.OPChildKeeper
}
```

6. IBC 통합

6.1 IBC Keeper 설정

```
// OPChild를 스테이킹 모듈로 사용하여 IBC Keeper 생성
appKeepers.IBCKeeper = ibckeeper.NewKeeper(
appCodec,
appKeepers.keys[ibcexported.StoreKey],
nil, // migration 불필요
appKeepers.OPChildKeeper, // 스테이킹 모듈 대신 사용
appKeepers.UpgradeKeeper,
appKeepers.ScopedIBCKeeper,
```

```
authorityAddr,
)
```

6.2 검증자 세트 업데이트

```
// IBC 클라이언트 업데이트 후 검증자 세트 동기화
appKeepers.IBCKeeper.ClientKeeper.SetPostUpdateHandler(
appKeepers.OPChildKeeper.UpdateHostValidatorSet,
)
```

7. Lane 시스템 통합

7.1 OPChild Lane 사용

```
// System Lane 매칭 핸들러
systemLane := initialanes.NewSystemLane(blockbase.LaneConfig{
// ... 설정
}, opchildlanes.SystemLaneMatchHandler())


// Free Lane 매칭 핸들러
freeLane := initialanes.NewFreeLane(blockbase.LaneConfig{
// ... 설정
}, opchildlanes.NewFreeLaneMatchHandler(app.ac, app.OPChildKeeper).MatchHandler(
```

7.2 수수료 검증 통합

```
// OPChild의 수수료 검증기 사용

txFeeChecker := opchildante.NewMempoolFeeChecker(app.OPChildKeeper).CheckTxFeeWi
```

8. Ante Handler 통합

8.1 OPChild Ante Decorator

```
// 중복 브리지 트랜잭션 방지

opchildante.NewRedundantBridgeDecorator(options.OPChildKeeper)
```

8.2 OPinit용 Ante Handler 생성

```
// OPinit 전용 Ante Handler 생성

ante.CreateAnteHandlerForOPinit(appKeepers.AccountKeeper, txConfig.SignModeHandl
```

9. API 통합

9.1 OPChild API 엔드포인트

Swagger 문서에서 확인할 수 있는 OPChild API들:

```
# 기본 denom 조회
```

```
/opinit/opchild/v1/base_denom


# 브리지 정보 조회
/opinit/opchild/v1/bridge_info


# L1/L2 시퀀스 번호 조회
/opinit/opchild/v1/next_l1_sequence
/opinit/opchild/v1/next_l2_sequence


# 파라미터 조회
/opinit/opchild/v1/params


# 검증자 정보 조회
/opinit/opchild/v1/validator/{validator_addr}
/opinit/opchild/v1/validators
```

9.2 OPChild 파라미터 구조

```go
type Params struct {
// 최소 가스 가격
MinGasPrices sdk.DecCoins
// 브리지 실행자 목록
BridgeExecutors []string
```

```
// 관리자 주소
Admin  string
// 후크 실행 최대 가스
HookMaxGas  uint64
}
```

10. 토큰 생성 통합

10.1 토큰 생성 함수 등록

```
// ERC20 Keeper의 토큰 생성 함수를 OPChild에 등록
appKeepers.OPChildKeeper.WithTokenCreationFn(erc20Keeper.TokenCreationFn)
```

10.2 자동 래핑 토큰 생성

```
func  generateOPBridgeHookMessage(
ctx launchtools.Launcher,
bridgeId  uint64,
operatorL2AddrStr  string,
operatorMnemonic  string,
) ([]byte, error) {
// 1. ERC20 Wrapper ABI 로드
wrapperABI, err := erc20_wrapper.Erc20WrapperMetaData.GetAbi()
// 2. L2 denom 계산
```

```
l2Denom := ophosttypes.L2Denom(bridgeId, INIT_DENOM)

// 3. ERC20 Wrapper 주소 조회

wrapperAddr, err := loadERC20WrapperAddress(*ctx.ClientContext())

// 4. toLocal 함수 호출 데이터 생성

input, err := wrapperABI.Pack("toLocal", common.BytesToAddress(operatorAddr), l2

// 5. EVM 호출 메시지 생성

msg := evmtypes.MsgCall{

Sender: operatorL2AddrStr,

ContractAddr: wrapperAddr.Hex(),

Input: hexutil.Encode(input),

Value: sdkmath.NewInt(0),

}

// 6. 트랜잭션 생성 및 서명

return  ctx.GetRPCHelperL2().CreateAndSignTx(operatorL2AddrStr, operatorMnemonic

}
```

11. 가스 가격 통합

11.1 OPChild 기반 가스 가격 Keeper

```
type opchildKeeperForGasPriceKeeper interface {

MinGasPrices(ctx context.Context) (sdk.DecCoins, error)

}
```

```go
type GasPriceKeeper struct {
opck opchildKeeperForGasPriceKeeper
}

func newGasPriceKeeper(opck opchildKeeperForGasPriceKeeper) GasPriceKeeper {
return GasPriceKeeper{opck: opck}
}
```

11.2 JSON-RPC에서 가스 가격 조회

```go
// JSON-RPC 백엔드에서 OPChild 파라미터 사용
params, err := b.app.OPChildKeeper.GetParams(queryCtx)
if err != nil {
return nil, err
}

// 최소 가스 가격 반환
return params.MinGasPrices, nil
```

12. 보안 및 권한 관리

12.1 브리지 실행자 권한

```go
// OPChild 파라미터에서 브리지 실행자 목록 관리
type Params struct {
BridgeExecutors []string  // 브리지 트랜잭션을 실행할 수 있는 주소들
Admin  string  // 파라미터를 변경할 수 있는 관리자 주소
}
```

12.2 권한 기반 모듈 계정

```go
// OPChild 모듈이 권한을 가진 모듈 계정으로 설정
authorityAccAddr := authtypes.NewModuleAddress(opchildtypes.ModuleName)
authorityAddr, err := ac.BytesToString(authorityAccAddr)
```

13. 모니터링 및 상태 추적

13.1 브리지 상태 정보

```go
type BridgeInfo struct {
BridgeId  uint64
L1ChainId  string
L1ClientId  string
L1ChannelId  string
BridgeConfig BridgeConfig
NextL1Sequence  uint64
```

```
NextL2Sequence  uint64
}
```

13.2 검증자 정보

```
type Validator struct {
OperatorAddress  string
ConsPubkey *types.Any
Jailed  bool
Status BondStatus
Tokens  string
DelegatorShares  string
Description Description
UnbondingHeight  int64
UnbondingTime time.Time
Commission Commission
MinSelfDelegation  string
Moniker  string
}
```

14. 결론

14.1 OPinit 통합의 핵심 가치

1. **완전한 Cosmos 통합**: IBC, 거버넌스, 스테이킹 등 모든 기능 활용

2. **자동화된 배포**: Launch Tools를 통한 원클릭 L2 배포

3. **검증자 없는 운영**: 복잡한 검증자 관리 불필요

4. **유연한 브리지**: 모든 Cosmos 토큰과 ERC20 간 브리징

14.2 기술적 혁신

1. **Lane 기반 처리**: 트랜잭션 타입별 최적화

2. **IBC 기반 브리지**: 안전하고 검증된 크로스체인 통신

3. **EVM 호환성**: 기존 Ethereum 도구와 완전 호환

4. **모듈식 아키텍처**: 필요에 따른 기능 확장 가능

MiniEVM은 OPinit 스택을 활용하여 Optimistic Rollup의 모든 장점을 제공하면서도 Cosmos 생태계의 풍부한 기능들을 완전히 통합한 혁신적인 L2 솔루션입니다.